DBP-CIM: Energy-Efficient 8T SRAM-Based Diagonal-Block Parallel Computing-in-Memory With Compact Data Layout for Arithmetic Operations

Dengfeng Wang, *Student Member, IEEE*, Chengjun Chang, *Student Member, IEEE*, Weifeng He, *Senior Member, IEEE*, Guanghui He, *Member, IEEE*, and Yanan Sun, *Senior Member, IEEE*

***Abstract*—In this work, an energy-efficient bit-parallel static random-access memory (SRAM)-based computing-in-memory (SRAM-CIM) is proposed for general-purpose in-memory arithmetic operations to adapt diverse computing tasks. A compact diagonal-block parallel (DBP) mapping scheme and a novel arithmetic flow are proposed to address the hardware underutilization issue in conventional two-sided stationary bit-parallel CIM architectures. Specifically, the DBP mapping method is implemented to enhance the throughput by reorganizing the intermediate and final results into diagonal memory blocks, effectively reducing the vacant CIM cells caused by the dynamic bit width during computing. In addition, the proposed hardware-efficient arithmetic flows employ: 1) a pipelined ADD scheme to reduce the critical path latency in near-memory computing units; and 2) shift-based arithmetic operations that halve the hardware resources required for multiplication and division while reducing energy consumption. The post-layout simulations on 28-nm CMOS technology show that the proposed DBP-CIM achieves higher energy efficiency and throughput for general-purpose arithmetic operations, compared with state-of-the-art works. Furthermore, evaluations on the general-purpose benchmarks demonstrate that the DBP-CIM reduces energy consumption and computing cycles by up to 55.9% and 60.9%, compared to the conventional bit-parallel CIM.**

***Index Terms*—Arithmetic operations, bit parallel, diagonal-data mapping, high energy efficiency, in-memory computing, static random-access memory (SRAM).**

I. Introduction

COMPUTING-IN-MEMORY (CIM) has been widely explored as one of the most promising solutions to break the memory wall in von Neumann architecture [1]. While nonvolatile memory-based CIM implementations [2], [3], [4], [5], [6] offer high density and low leakage power, volatile static random-access memory (SRAM)–CIM is becoming an attractive alternative because of its superior speed and technological maturity. Most current SRAM-CIM are designed for AI tasks and typically optimized for massive parallel matrix–vector multiplications through a one-sided stationary data flow. In this approach, static weights are stored in the CIM array, while the dynamic activations are streamed in for computation, matching the distinct characteristics of each data type. By leveraging the inherent analog properties of memory arrays [7], [8], [9], [10], integrating digital computing circuits [11], [12], [13], [14], [15], [16], [17], or employing hybrid computing methods [18], SRAM-CIM achieves high energy efficiency and throughput for domain-specific tasks. Recent advances have further improved the bit flexibility [12], [13], [14], [15], [16] of SRAM-CIM and extended its computing precision through floating point [17], [18], [19], [20] and emerging coding schemes [21], thereby enabling higher accuracy in practical applications.

While CIM processors continue to flourish as efficient AI accelerators, recent research has begun exploring multifunctional CIM architectures [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], [32], [33] for general-purpose computing. These versatile CIM macros offer significant advantages by enabling task reuse across different applications, making them particularly suitable for heterogeneous acceleration that depend on reconfigurable computing, an area attracting growing attention from both academia and industry [27], [36]. The key to achieve functional flexibility in CIM lies in designing macros that support fundamental operations, including Boolean logic [22], [23], [24], [25], [26], [35] and arithmetic computations [27], [28], [29], [30], [31], [32], [33], [34]. This capability allows CIM to extend beyond neural networks, supporting a wide range of applications, such as database queries [25], [26], image processing [29], [33], and cryptographic operations [23], [24] in edge computing with stringent demands for power [37].

Current CIM arrays implement general Boolean and arithmetic operations through either bit-serial [29], [30] or bit-parallel [31], [32], [33], [34] processing approaches. The typical bit-serial and bit-parallel SRAM-CIM architectures for Boolean logic and arithmetic operations are shown in Fig. 1(a) and (b), respectively, which employ a two-sided

Received 9 July 2025; revised 10 September 2025 and 24 September 2025; accepted 25 September 2025. Date of publication 8 October 2025; date of current version 30 December 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 92464302 and Grant 62174110, in part by the National Key Research and Development Program of China under Grant 2021YFA0717400, in part by the Natural Science Foundation of Shanghai under Grant 23ZR1433200, and in part by Shanghai 2024 "Science and Technology Innovation Action Plan" Fundamental Research Program in Integrated Circuit under Grant 24JD1400300. *(Dengfeng Wang and Chengjun Chang are contributed equally to this work.) (Corresponding author: Yanan Sun.)*

The authors are with the Department of Micro-Nano Electronics, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: yungsk@sjtu.edu.cn; changchengjun@sjtu.edu.cn; hewf@sjtu.edu.cn; guanghui.he@sjtu.edu.cn; sunyanan@sjtu.edu.cn).

Digital Object Identifier 10.1109/TVLSI.2025.3615793

1063-8210 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:42 UTC from IEEE Xplore. Restrictions apply.

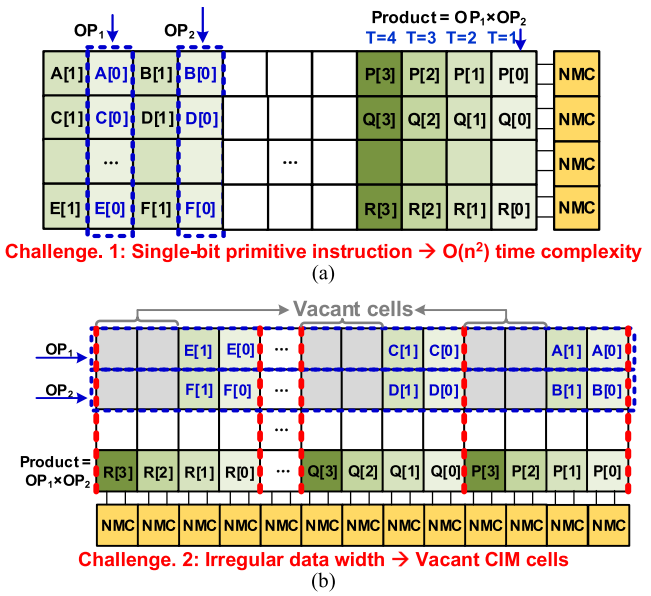


Fig. 1. Challenges of previous SRAM-CIM for general-purpose arithmetic. (a) Bit-serial SRAM-CIM [29]. (b) Bit-parallel SRAM-CIM [31], [32], [33], [34].

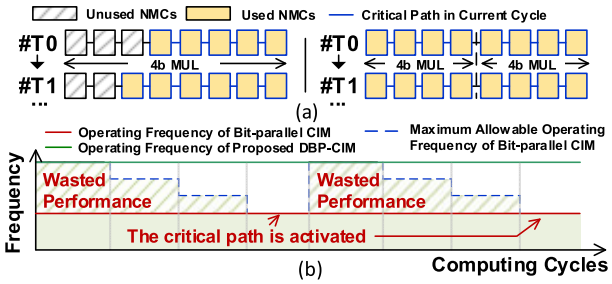


Fig. 2. Comparisons of previous bit-parallel CIM [31], [32], [33], [34] and the proposed DBP-CIM: (a) hardware usage and (b) performance, where previous bit-parallel works suffer from underutilized NMCs and wasted performance due to the irregular data width during computing.

TABLE I
COMPARISONS OF TWO-SIDED STATIONARY CIM WORKS FOR GENERAL-PURPOSE COMPUTING

	Conventional Bit-Serial CIM [29], [30]	Conventional Bit-Parallel CIM [31]-[34]	**Proposed Diagonal-Block Parallel CIM (DBP-CIM)**
Hardware Utilization	☺	☹	☺
Cycle Latency	☺	☹	☺
Computing Complexity	☹	☺	☺

stationary data flow. Unlike previous one-sided stationary CIM [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], this approach stores all the operands within memory array, enabling efficient support for computing flows beyond neural networks with reduced data transfer.

In the conventional bit-serial processing flow [29], one data bit corresponding to the two operands will be simultaneously selected and read to generate Boolean logic results on bitlines (BLs). The logic results are then sensed and processed by the near-memory computing (NMC) units with configurable functions, which are independently attached to each computing rows. The computed temporal or final results of NMC are then written back into the memory array for subsequent processing. The bit-serial approach enables bit-wise operations, offering inherent advantages for variable bit-width processing [15], [16], [20], [29], [30] and supporting complex data formats, including floating point [20], [29]. However, in order to select an arbitrary data bit in a word, the bits of data stored in the CIM array have to be aligned along columns [15], [16], which is incompatible with conventional bit-parallel data format. Several bit-serial architectures introduce transposable CIM cells [20], [29] for supporting bit-wise operations while suffering from storage density loss. Besides, due to the bit-wise operating nature, the bit-serial SRAM-CIM processes one bit of multiple data per cycle. Thus, it suffers from $O(n)$ time complexity for addition (ADD) and $O(n^2)$ for multiplication, which makes it long latency for a single task. Specifically, to conduct a 32-bit floating point arithmetic, bit-serial architecture can consume more than 3000 cycles [29].

The bit-parallel architecture stores operands in bit-parallel manner, where the data bits of operands with same significance are aligned in the same column, as shown in Fig. 1(b). Unlike the bit-serial one, the bit-parallel architecture processes all the bits of operands in a single cycle within the NMC units attached to each memory column. It therefore achieves low time complexity for ADD and multiplication. Previous bit-parallel CIM [31], [32], [33], [34] typically employs relatively simple 6T or 8T SRAM-CIM cells to read out all bits of two operands simultaneously via highly capacitive BLs, after which these values are sent to NMC units for further processing. To support bit-parallel computation, the NMC units are interconnected, allowing bit-significance information to propagate between adjacent columns. However, this interconnection introduces a long critical path that scales with the operating bit width. As a result, both BL discharge latency and NMC propagation delay become critical and tend to dominate the total cycle time. Lee et al. [31] use a BL separator to reduce logic computing latency, while the near-memory computing latency has not been addressed. Simon et al. [33] reduce cycle period by using a Manchester carry chain with the advanced carry signals to break the long ripple path among NMCs, while it introduces hardware complexity and lacks bit-width flexibility. Besides, to support multiprecision tasks, the vacant SRAM-CIM cells are required to be reserved to accommodate the intermediate data with irregular bit width in previous bit-parallel SRAM-CIM [31], [32], [33], [34]. Agrawal et al. [34] struggles to handle different bit-precision requirements in bit-parallel CIM by using staggered mapping; however, it lacks the expendability to complex arithmetic like multiplication and division. As a result, these vacant SRAM-CIM cells remain unused during computing, leading to wasted hardware resources and increased energy consumption. Furthermore, the previous bit-parallel arithmetic methods [31], [32], [33] typically need add-and-shift operations for multiplication and division. Consequently, the redundant NMC operations are required where the zero-padded operands need to be iteratively added with the partial sums, leading to reduced energy efficiency.

The comparisons between previous two-sided stationary computing methods for general-purpose computing are listed in Table I. The conventional bit-serial or bit-parallel manner cannot achieve high hardware utilization, low cycle latency, and low computing complexity simultaneously. To address the challenges above, an 8T bit-parallel SRAM-CIM array

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:42 UTC from IEEE Xplore. Restrictions apply.

with diagonal-block parallel (DBP) mapping method and hardware-efficient arithmetic flows is proposed to enhance energy efficiency in general-purpose arithmetic operations. The proposed DBP-CIM features higher hardware efficiency and higher operating frequency compared to previous bit-parallel CIM, as shown in Fig. 2. The proposed DBP-CIM utilizes only one additional state flip-flop in each NMC unit to temporarily store the frozen low-bit product of partial sum, thereby avoiding the redundant NMC operations with enhanced hardware utilization and energy efficiency. The main contributions of this work are summarized as follows.

1) A DBP mapping method is proposed for improving hardware utilization and throughput by splitting and storing the data in a diagonal-block form to reduce the vacant SRAM-CIM cells caused by the irregular bit width during computation.

2) The optimized arithmetic flows based on DBP mapping method are proposed with reduced critical path delay and hardware resources to enhance the computing throughput and energy efficiency.

3) An 8T SRAM-based CIM macro with customized peripheral circuits is designed for supporting general-purpose arithmetic with the proposed DBP mapping method and arithmetic flows.

4) The evaluations demonstrate that the proposed DBP-CIM outperforms previous bit-parallel CIM works in energy efficiency and throughput, as demonstrated by the post-layout simulations under 28-nm CMOS technology. The proposed DBP-CIM also achieves a maximum reduction of 55.9% in energy consumption and 60.9% in computing cycles on general-purpose computing tasks compared to the conventional bit-parallel CIM baseline.

This article is organized as follows. The overall architecture of the proposed DBP-CIM is introduced in Section II. The diagonal mapping method and arithmetic operation flows are presented in Section III. Simulations and comparisons are presented in Section IV. Finally, conclusions are provided in Section V.

II. Proposed DBP-CIM Macro

This section presents DBP-CIM with customized peripheries for supporting DBP arithmetic operations. The overview of the proposed DBP-CIM macro is introduced in Section II-A. The near-memory computing units and primary cycle operations are presented in Section II-B.

A. Overview of the Proposed DBP-CIM

The proposed DBP-CIM macro is shown in Fig. 3, where the macro is composed by the 128 × 128 8T SRAM-CIM arrays, BL separators, row decoders, customized column peripheries, including sense amplifiers (SAs), write-back drivers, near-memory computing circuits, and state-flip-flops (S-FFs). Specifically, the row decoders select the intended data rows for reading, writing, and computing. The BL separators are used to divide the 8T SRAM-CIM array into the upper and lower parts, where the upper part stores operands or final results while the lower part consists of eight temporary rows for temporary results. Depending on the operations in the current cycle, the BL separators conditionally disconnect the differential read BLs (RBL and RBLB) within the SRAM-CIM array to reduce energy consumption caused by charging and discharging the large capacitive BLs. Each column in the 8T SRAM-CIM array is connected with SAs and write-back drivers, which facilitate parallel read and write operations, respectively. The NMC unit followed with the SA processes the output (OUT) data for subsequent operations. The extra S-FFs record the computing state in each CIM cycle to configure the data path of NMCs for implementing specific arithmetic operations.

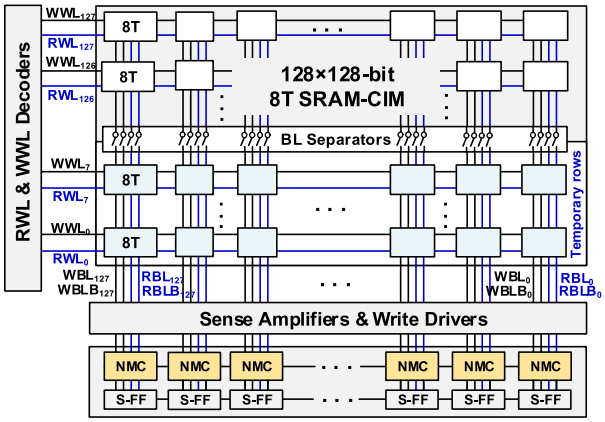
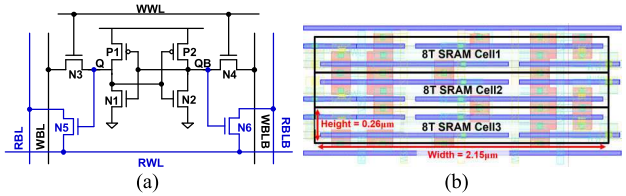


Fig. 3. Proposed DBP-CIM macro.



(a) (b)

Fig. 4. Differential 8T SRAM-CIM cell. (a) Schematic. (b) Layout.

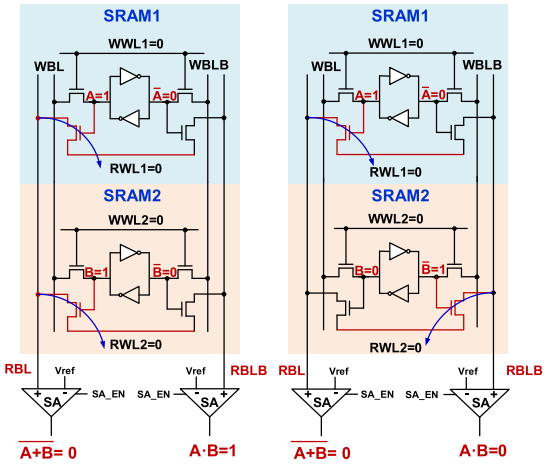
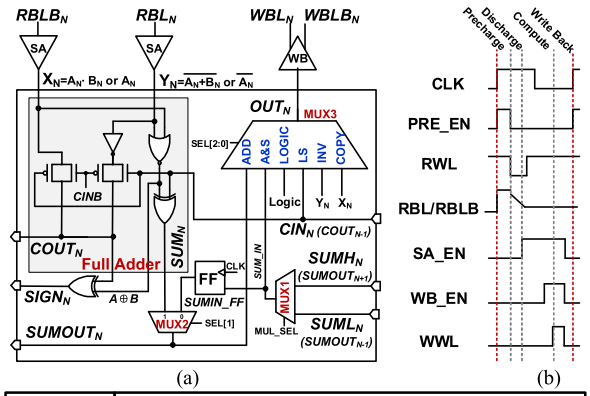


Fig. 5. In-memory logic operations of differential 8T SRAM-CIM cells.

The 8T SRAM-CIM cells [32] used in this work and their corresponding layout are illustrated in Fig. 4. This cell supports both memory and CIM modes. In-memory mode, it performs conventional read and write operations. To prevent read disturbances during simultaneous row activation, the 8T SRAM employs two independent data access paths: one for write operations (N_3 and N_4) and another for read operations (N_5 and N_6). For write operations, data bits are applied to the

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:42 UTC from IEEE Xplore. Restrictions apply.



(a) (b)

	Supported Primitive Operations
LS	Shifting a single operand left
COPY	Copying the current row to a new row
INV	Single operand inversion
LOGIC	Logic operations between two operands
A&S	Shifting the sum result of current or previous cycle left/right
ADD	Adding two operands

(c)

Fig. 6. (a) Schematic of near-memory computing units. (b) Timing diagram. (c) Primitive operations of near-memory computing units.

TABLE II
SELECTION CODES OF NEAR-MEMORY COMPUTING UNIT

SEL [2:0]	SUMOUT	OUT	Functions
000	SUMIN_FF	SUMOUT	Output the sum of last cycle
010	SUM	SUMOUT	Output the sum of current cycle
001	SUMIN_FF	SUMH(L)	Shift right (left) sum of last cycle
011	SUM	SUMH(L)	Shift right (left) sum of current cycle
100	SUMIN_FF	X	Output X (or AND)
101	SUMIN_FF	CIN	Shift one bit left
110	SUM	Y	Output X_bar (or NOR)
111	SUM	LOGIC	Output XOR

complementary write BLs (WBL and WBLB) and driven into the storage nodes (Q and QB) upon activation of the write wordline (WWL). For read operations, the differential read BLs are first precharged and then conditionally discharged based on the stored data when the read wordline (RWL) is active-low. In CIM mode, two 8T SRAM-CIM cells aligned in the same column are activated simultaneously through read operations, producing NOR and AND logic results on RBL and RBLB respectively, which are then detected by strong-arm SAs connected to each RBL, as shown in Fig. 5. Thanks to the decoupled read and write paths, the 8T SRAM-CIM cell has a read static noise margin of 0.317 V, which is 0.194 V higher than the standard 6T SRAM, without data corruption concern in CIM mode.

B. Proposed Peripheral Computing Circuit

The proposed NMC consists of an in-memory full adder, three multiplexers (MUXs), and one flip-flop for storing the sum (SUMIN_FF), as shown in Fig. 6(a). The full adder is designed to perform Boolean logic operations for single-bit ADD. The three MUXs controlled by selection codes SEL[2:0] are used to configure the data path to realize various NMC functions. Specifically, MUX_1 connects the neighboring *SUMOUT* nodes, allowing right- and left-shift operations in multiplication and division, respectively. MUX_2 determines

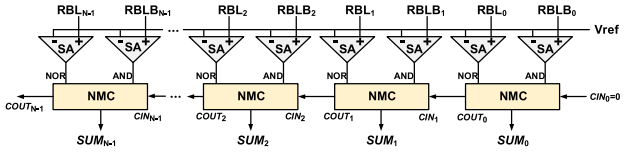


Fig. 7. Near-memory computing with rippled data path.

whether the OUT is the ADD results of the current cycle or the latched results in the SUMIN_FF. MUX_3 selects among different primitive operations and transfer the intended *OUT* to the write-back circuit. These primitive operations include row copying (COPY), single-bit inversion (INV), left shifting (LS), Boolean logic operations (logic XOR), right/LS the ADD result of the current or previous cycle (A&S), and ADD. Note that, the basic logic operations like NOR and AND can also be supported by activating two rows and selecting the nodes COPY and INV of MUX_3, respectively. The detailed operating codes for the primitive operations of NMC are listed in Table II.

The timing diagram of a single CIM cycle is shown in Fig. 6(b). The CIM cycle begins with enabling the precharge circuit, which charges RBL and RBLB to V_{DD}. Subsequently, one or two rows of SRAM-CIM cells are selected by pulling down RWL. Once RBL and RBLB discharge in response to the lowered RWL, SA_EN is raised to activate the SAs. The AND and NOR results of the two operands are generated by the SAs and fed into the NMC, which executes the intended primitive operations. The adjacent NMCs are connected and ripple information across different CIM columns to enable bit-parallel processing, as shown in Fig. 7. Upon completion of the computation within NMCs, the write-back circuit is activated, while WWL is raised to write the computing result back to the specified row. By combining multiple primitive operations across multiple CIM cycles, the basic Boolean logic and arithmetic operations like subtraction, accumulation, multiplication, and division can be effectively implemented.

III. PROPOSED DBP MAPPING METHOD AND ARITHMETIC OPERATIONS

To address the hardware underutilization issue and boost the computing parallelism in the previous bit-parallel CIM architecture, a novel DBP mapping method and its corresponding arithmetic flows are proposed in this section. The overview of the proposed DBP mapping method and arithmetic operations is presented in Section III-A. The detailed arithmetic flows based on the proposed DBP mapping method are presented in Sections III-B–III-D.

A. Overview of the Proposed DBP Mapping Method and Arithmetic Operations

Previous bit-parallel SRAM-CIM suffers from underutilized hardware due to the irregular bit width of intermediate computing results, as shown in Fig. 8(a). Using 2-bit multiply-and-accumulate (MAC) operations for example, the initial 2-bit operands (OP_1–OP_4) are expanded to 5-bit results (S_1) after 2-bit multiplications and 4-bit ADD. To satisfy the bit-significance alignment requirement of bit-parallel SRAM-CIM, every five columns have to be formed as a 2-bit MAC unit, while the original operands only occupy two columns

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:42 UTC from IEEE Xplore. Restrictions apply.

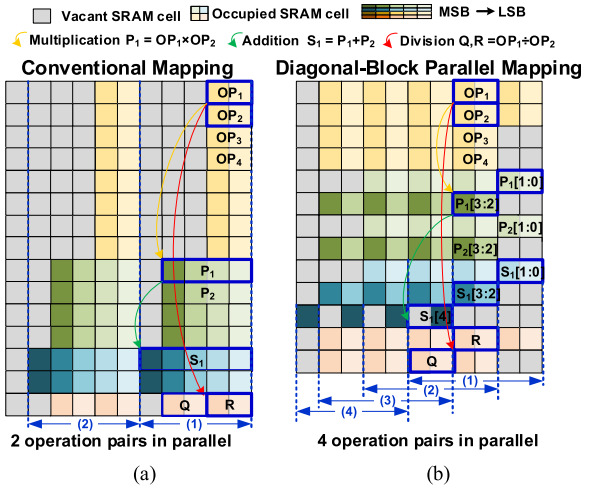


Fig. 8. Overview of conventional bit-parallel mapping and the proposed DBP mapping for different arithmetic operations. (a) Conventional bit-parallel mapping. (b) Proposed diagonal-data mapping.

with three columns prefilled with zeros. This irregular data format generated during a sequence of computing tasks leads to two problems. First, the overall operands are less stored in the SRAM-CIM array, with underutilized capacity. Second, increasing the operating bit width during computation requires reserving sufficient NMCs for high-precision operands, leading to NMC idleness in the initial computing stages.

To overcome the hardware underutilization caused by dynamic bit-width requirements in conventional bit-parallel CIM arrays, this work introduces a compact DBP mapping. The basic idea of DBP mapping is to partition the dynamic bit width of intermediate and final results into multiple uniform bit blocks, making it feasible to seamlessly place them into the CIM array in a diagonal manner like Tetris. As illustrated in Fig. 8(b), the proposed DBP mapping achieves higher storage utilization, thereby enabling more operands stored within the CIM arrays of identical sizes, compared to traditional bit-parallel mapping scheme.

The proposed arithmetic flows based on DBP mapping can further effectively reduce the NMC idleness during arithmetic operations, consequently improving computing throughput. As demonstrated in Fig. 8(b), the proposed DBP mapping-based arithmetic flows are specifically explained with multiplication and accumulation operations for simplicity. For multiplication, the 4-bit product of two 2-bit operands (OP_1 and OP_2) is split into two bit blocks with 2-bit length and stored across adjacent rows using DBP mapping. Unlike conventional bit-parallel arithmetic, which requires zero padding to match the product length, the proposed method processes only the fixed-length bit blocks, reducing the required NMCs from 4 to 2 for the same operation. This optimization doubles computing throughput for a given CIM array size. The DBP mapping-based accumulation follows the similar principle of multiplication that only operates in a granularity of bit block instead of the full-bit length of final results. For example, computing $S_1 = P_1 + P_2$ (where P_1 and P_2 are diagonally stored products across two bit blocks) takes two cycles, with each cycle summing 1-bit block. The resulting 5-bit sum S_1 is therefore split into three blocks (S_1 [4], S_1[3:2], and S_1[1:0], where S_1 [4] is the overflow carry bit) and stored

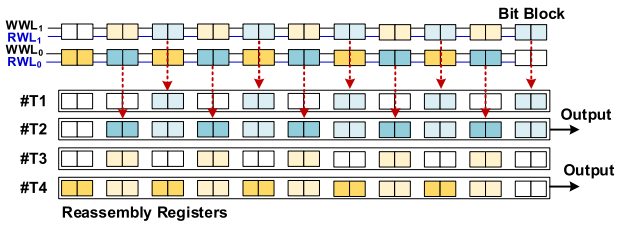


Fig. 9. Readout scheme of results stored in diagonal-block format.

across three consecutive rows. This dense storage scheme maximizes computing parallelism and NMC utilization within the CIM array. Subsequent accumulation between S_1 and S_2 will dynamically expand the storage by shifting overflow bits into adjacent diagonal bit blocks with fixed length. The dummy bit blocks induced by the proposed DBP mapping method have minimal impact on performance, as they are only confined to the edge of array. Sections III-B–III-D will detailly introduce the operation flows of the proposed DBP-based arithmetic, including ADD (or pipelined ADD), subtraction, multiplication, and division.

Since the final results are distributed across diagonally placed bit blocks, k^2 cycles [complexity of $O(k^2)$] are required to OUT and reassemble the data into conventional bit-parallel format, where k represents the number of rows spanned by the bit blocks, and we assume that there are k sets of data in a row for simplicity. For example, reading two sets of diagonally mapped data across two rows requires four cycles, as shown in Fig. 9. Specifically, for every two cycles, one set of results are read and reassembled, until all the results (lighted in blue and yellow, respectively) are OUT. Using k sets of parallel reassembly registers can further reduce the complexity into $O(k)$. Note that, the reassembly operation (only once for each final result data) has little impact on the total execution cycles, which are dominated by the extensive computations in CIM array. Besides, thanks to the diagonal mapping scheme, the performance of the proposed DBP-CIM can be still maintained high despite of the reassembly overhead.

B. Conventional/Subtraction and Pipelined Addition

The proposed DBP-CIM supports both conventional bit-parallel ADD, subtraction, and diagonal pipelined ADD, accommodating both signed and unsigned number, as shown in Fig. 10.

1) Conventional Bit-Parallel ADD and Subtraction: The conventional n-bit ADD takes one computing cycle to get n-bit SUM[$n-1$:0] and 1-bit COUT in unsigned ADD or SIGN in signed ADD, as shown in Fig. 11. A total of the $(n+1)$th columns of SRAM-CIM cells and their corresponding NMC units are needed in conventional n-bit ADD. The lower n NMC units (corresponding to the n-bit operands) are cascaded to form a ripple-carry adder, while the $(n+1)$th NMC unit is used to write-back either the carry OUT or the sign extension bit generated from the nth NMC to the CIM array. The sign bit SIGN can be expressed as

$$\mathrm{SIGN}_{n-1} = A[n-1] \oplus B[n-1] \oplus \mathrm{COUT}_{n-1}$$

where $A[n-1]$ and $B[n-1]$ are the most significant bits of the operands in ADD. The near-memory computing unit gets SIGN_{n-1} by using the embedded XOR gate.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:42 UTC from IEEE Xplore. Restrictions apply.

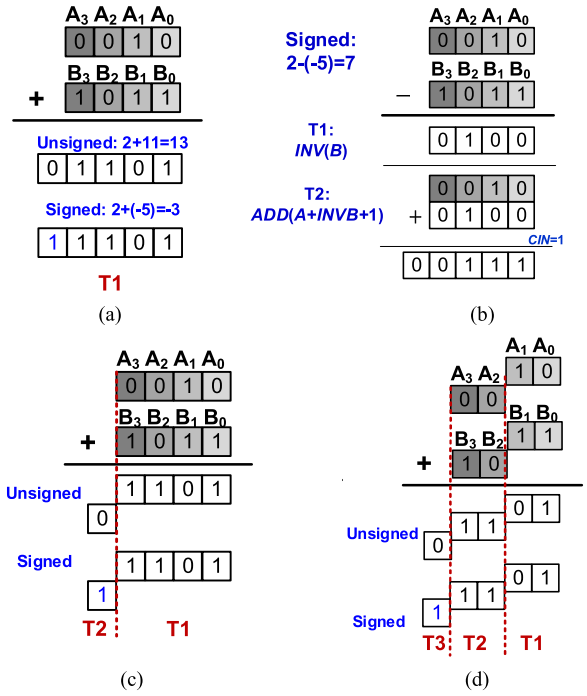


Fig. 10. Supported (a) conventional ADD and (b) subtraction in the proposed DBP-CIM. The proposed DBP-CIM also supports pipelined ADD with the initial data layout to be organized as (c) conventional bit-parallel and (d) proposed DBP. Note that, the signed operations need extra sign extension, which is not considered in previous bit-parallel works [31], [32], [33], [34].

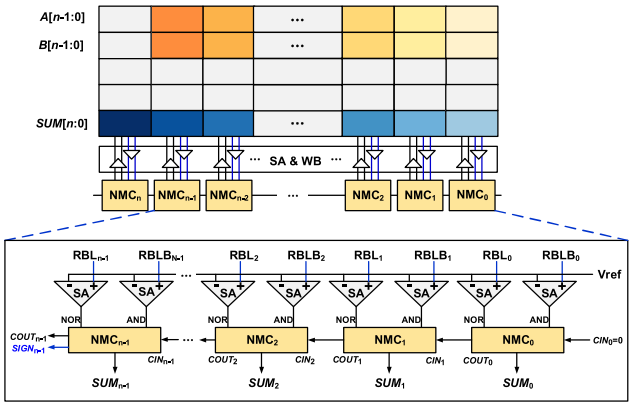


Fig. 11. Example of the proposed bit-parallel ADD flow.

Subtraction is similar to ADD, as the subtraction of two signed numbers A and B is equivalent to performing signed ADD between A and the negation of B (−B). The two's complement representation of −B is obtained by inverting all bits of B and then adding "1." Therefore, conventional signed subtraction requires two cycles, as shown in Fig. 10(b). In the first cycle, each bit of the subtrahend B is inverted and written back to the CIM array. To implement the INV, the following operations are executed: 1) the SRAM row containing B is activated; 2) all bits are read into their corresponding NMC units; and 3) the control signal SEL[2:0] is set to 110 to route the inverted bits to the OUT for write-back operation. In the second cycle, both the inverted subtrahend and the minuend rows are activated simultaneously. The carry input (CIN_0) of NMC connected with the least significant bit is set to "1" to conduct n-bit ADD.

*2) **Pipelined ADD:*** The pipelined ADD is composed of multicycle ADD and primarily used for scenarios involving ADD of multiple data. A k*n-bit pipelined ADD needs k computing cycles to finish and an additional cycle may be needed when there is overflow carry bit, where k is the number of diagonal bit blocks and n is the bit width of each bit block, as shown in Fig. 10(c) and (d).

When k is 1, the operands are initially in conventional bit-parallel data layout, as shown in Fig. 10(c). Two cycles are needed to generate the sum as DBP format. In the first cycle, n-bit ADD is performed with the n-bit sum written back to the SRAM-CIM array. Simultaneously, the S-FF is used to latch the carry OUT of the NMC corresponding to the most significant bit. In the second cycle, the carry OUT latched in S-FF is written back to the next row of n-bit sum as the most significant bit of the final result. The diagonal-block data layout is formed, as the $(n+1)$th bit is diagonally placed relative to the lower n bits.

When k is larger than 1, k^*n-bit pipelined ADD requires k (without overflow carry bit) or $k+1$ (with overflow carry bit) cycles to process multiple diagonally placed n-bit blocks, as shown in Fig. 10(d). During the ith computing cycle, the ADD between bit blocks storing bits $A[n^*(i+1)-1: n^*i]$ and $B[n^*(i+1)-1: n^*i]$ is performed. The sum is written back to the SRAM-CIM array, while the carry OUT of current bit block is latched by using an S-FF. The subsequent $(i+1)$th cycle utilizes the latched carry from the S-FF as carry input to compute the ADD of the next set of bit blocks, which store $A[n^*(i+2)-1: n^*(i+1)]$ and $B[n^*(i+2)-1: n^*(i+1)]$. The resulting sum is again written back to the CIM array with the new carry OUT stored in the S-FF. This process continues iteratively until all k diagonal bit blocks are processed.

In summary, a k^*n-bit pipelined ADD computes and writes back n-bit bit block cycle by cycle. When the bit width of result exceeds the bit length of bit block, the overflow bits will be written back to the diagonal bit block in next row in the following cycle. By eliminating the need to reserve dummy memory columns, the rows beyond the ADD result rows can be well utilized; thus, the overflow bit blocks have a negligible impact on the cell utilization of CIM array, as shown in Fig. 8(b). Besides, the pipelined ADD ensures that even when the bit width of accumulated data increases, the length of the carry delay chain formed by cascaded NMCs remains unchanged. The latency of critical path is limited only to the delay of processing a single diagonal bit block. Compared to traditional ADD where longer bit width leads to longer single-cycle computing delay, this pipelined ADD maintains a constant single-cycle computing period, featuring constant operating frequency overtime without performance degradation caused by the gradually increased critical path.

C. Right-Shift Multiplication Operation

The proposed DBP-CIM supports both unsigned and signed multiplication with reduced hardware consumption by implementing a right-shift-based computing flow. The key innovation of the proposed multiplication flow lies in strategically omitting computations for the frozen low-bit products that would normally be processed in conventional bit-parallel arithmetic, thereby substantially decreasing the required number of NMCs, as shown in Fig. 12.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:42 UTC from IEEE Xplore. Restrictions apply.

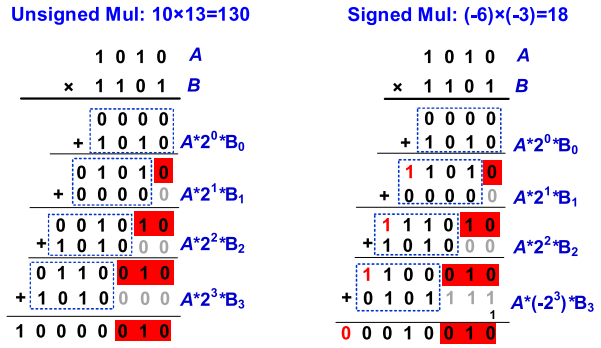


Fig. 12. Example of conventional left-shift-based multiplication [31], [32]. Previous left-shift-based multiplication processes all bits of partial sum each cycle, while the low bits highlighted in red are not changed after being calculated. The low bits in red are referred as frozen bits in this work.

Algorithm 1 Right-Shift-Based Unsigned Multiplication

Input: A[n-1:0], B[n-1:0] // Multiplicand A and Multiplier B
Output: P[2n-1:0] //Product P= Mul (A, B)

0: Initialization // Store B in S-FF [n-1:0]; initialize P[2n-1:0] and SUMIN_FF[n-1:0] to 0
1: **for** (i =0 to n-1) **do**
2: {C_{OUT}, SUM[n-1:0]}=ADD(A, P[2n-1:n],1'b0)
3: **if** S-FF[0]== 1'b1 **then** // B[i]== 1'b1; add and right shift
4: S-FF={SUM[0], S-FF[n-1:1]} //Save SUM[0] in S-FF
5: P[2n-1:n]={ C_{OUT}, SUM[n-1:1]} //Right shift the partial sum
SUMIN_FF[n-1:0]= {C_{OUT}, SUM[n-1:1]}
6: **else** // B[i]== 1'b0; right shift the partial sum of the previous cycle
7: S-FF={ SUMIN_FF[0], S- FF[n-1:1]} //Save P[n] in S-FF
8: P[2n-1:n] = {1'b0, SUMIN_FF[n-1:1]}
9: **end if**
10: **end for**
11: P={P[2n-1:n], S-FF[n-1:0]}//The less significant part stored in S-FF
12: **Return P**

1) Right-Shift-Based Unsigned Multiplication: The computing flow of the proposed right-shift-based unsinged multiplication is shown in Algorithm 1. Initially, the n-bit operands A and B are stored in CIM array, and the product results $P[2n-1:0]$ are initialized to 0. To start the multiplication, the multiplier B is first loaded into the S-FF to configure the NMC data paths for subsequent operations. For the next n computing cycles, the multiplicand $A[n-1:0]$ and the half temporal result $P[2n-1:n]$ are read out to conduct unsigned ADD. In each computing cycle, the current ADD results are conditionally right-shifted and written back according to the state of S-FF[0]. If S-FF[0] is "1," the ADD result in current cycle is selected. The low bit SUM[0] is treated as a frozen bit. S-FF$[n-1:0]$ is right-shifted, with SUM[0] taking the position of S-FF$[n-1]$. The carry COUT and SUM$[n-1:1]$ are written back to $P[2n-1:n]$ and update the value latched in SUMIN_FF. If S-FF[0] is "0," the ADD result in current cycle is abandoned. The sum stored in the SUMIN_FF$[n-1:0]$ of previous cycle is selected, right-shifted with S-FF$[n-1:0]$, and written back to $P[2n-1:n]$.

The specific computing example of unsigned multiplication between operand A "1010" and operand B "1101" is shown in Fig. 13(a). The operands A and B are stored and aligned in columns 4–7, while the columns 0–3 store another set of operands with its own S-FF. The first computing cycle is used to store operand B in S-FF connected with columns 4–7. In the following cycles, operand A and partial sums are repetitively added. The ADD results are right-shifted with S-FF. The state of S-FF[0] determines the data path in NMCs. When S-FF[0] is "1," the add-and-right-shift path is activated. When S-FF[0] is "0," the data stored in SUMIN_FF are selected and right-shifted. Finally, product "10000010" is generated with lower bits in S-FF connected with columns 0–3 and higher bits in columns 4–7. The higher bits and lower bits are then written into CIM arrays in a diagonal-block format.

Algorithm 2 Right-Shift-Based Signed Multiplication

Input: A[n-1:0], B[n-1:0] // Multiplicand A and Multiplier B
Output: P[2n-1:0] //Product P= Mul (A, B)

0: Initialization //Store A and B in the array; initialize P[2n-1:0] to 0
1: Read B and store B[n-1:0] in S-FF[n-1:0]
2: Temp[n-1:0] = INV(A[n-1:0]) // Invert A and write it to one temporary row
3: **for** (i =0 to n-2) **do** //Process B[n-2:0]
4: {SIGN, SUM[n-1:0]}=ADD(A, P[2n-1:n],1'b0)
5: **if** S-FF[0]== 1'b1 **then** // B[i]=1'b1; add and right shift with sign bit
6: S-FF={SUM[0], S-FF[n-1:1]} //Save SUM[0] in S-FF
7: P[2n-1:n]={SIGN, SUM[n-1:1]} //Right shift the partial sum
8: **else** // B[i]= 1'b0; right shift the partial sum of the previous cycle with sign bit
9: S-FF={P[n], S- FF[n-1:1]} //Save P[n] in S-FF
10: P[2n-1:n] = { P[2n-1], P[2n-1:n+1]}
11: **end if**
12: **end for**
13: {SIGN, SUM[n-1:0]}=ADD(Temp, P[2n-1:n],1'b1) // Calculate the multiplication between A and MSB of B
14: **if** S-FF[0]== 1'b1 **then** // B is negative; add and right shift with sign bit
15: S-FF={SUM[0], S-FF[n-1:1]} //Save SUM[0] in S-FF
16: P[2n-1:n]={ SIGN, SUM[n-1:1]} //Right shift the partial sum
17: **else** // B is positive; right shift the partial sum of the previous cycle with sign bit
18: S-FF={P[n], S- FF[n-1:1]} //Save P[n] in S-FF
19: P[2n-1:n] = {P[2n-1], P[2n-1:n+1]}
20: **end if**
21: P={P[2n-1:n], S-FF[n-1:0]} //The less significant part stored in S-FF
22: **Return P**

2) Right-Shift-Based Signed Multiplication: The right-shift-based signed multiplication flow between multiplicand $A[n-1:0]$ and multiplier $B[n-1:0]$ is shown in Algorithm 2. The algorithm begins with storing $B[n-1:0]$ in S-FF, while inverting A and writing it to one temporal row as Temp$[n-1:0]$ in two cycles. The algorithm decomposes the signed multiplication into two parts. In the first part, it takes the $(n-1)$th cycles to conduct right-shift-based multiplication between the signed $A[n-1:0]$ and unsigned part of $B[n-2:0]$, where the ADD results between $A[n-1:0]$ and partial product $P[2n-1:n]$ in each multiplication cycle are repetitively right-shifted and written back or not, according to the states of S-FF[0], similar to the operations in unsigned multiplication. Since $A[n-1]$ and $P[2n-1:n]$ are both signed number, the carry signal generated in unsigned multiplication flow is XOR with the results of A XOR B to generate the extended sign bit of ADD. In the second part, the algorithm operates on the signed $A[n-1:0]$ and signed bit of $B[n-1]$. The Temp$[n-1:0]$ is added with $P[2n-1:n]$ with input carry signal set to "1," which stands for a subtraction between $A[2n-1:0]$ and $P[2n-1:n]$. If $B[n-1]$ is "1," the subtraction is valid and followed with right-shift and write-back operations; otherwise, the ADD result in previous cycle is selected for a right-shift operation along with S-FF$[n-1:0]$ and then written back to $P[2n-1:n]$.

The specific computing example of signed multiplication between operand A "1010" and operand B "1101" is shown

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:42 UTC from IEEE Xplore. Restrictions apply.

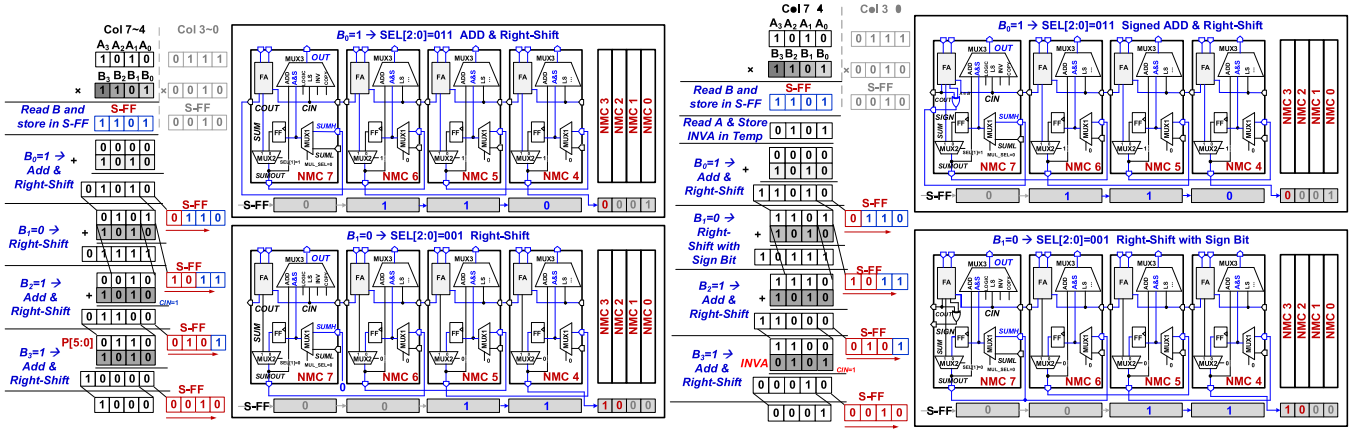


Fig. 13. Proposed (a) unsigned multiplication and (b) signed multiplication flow for supporting DBP data format.

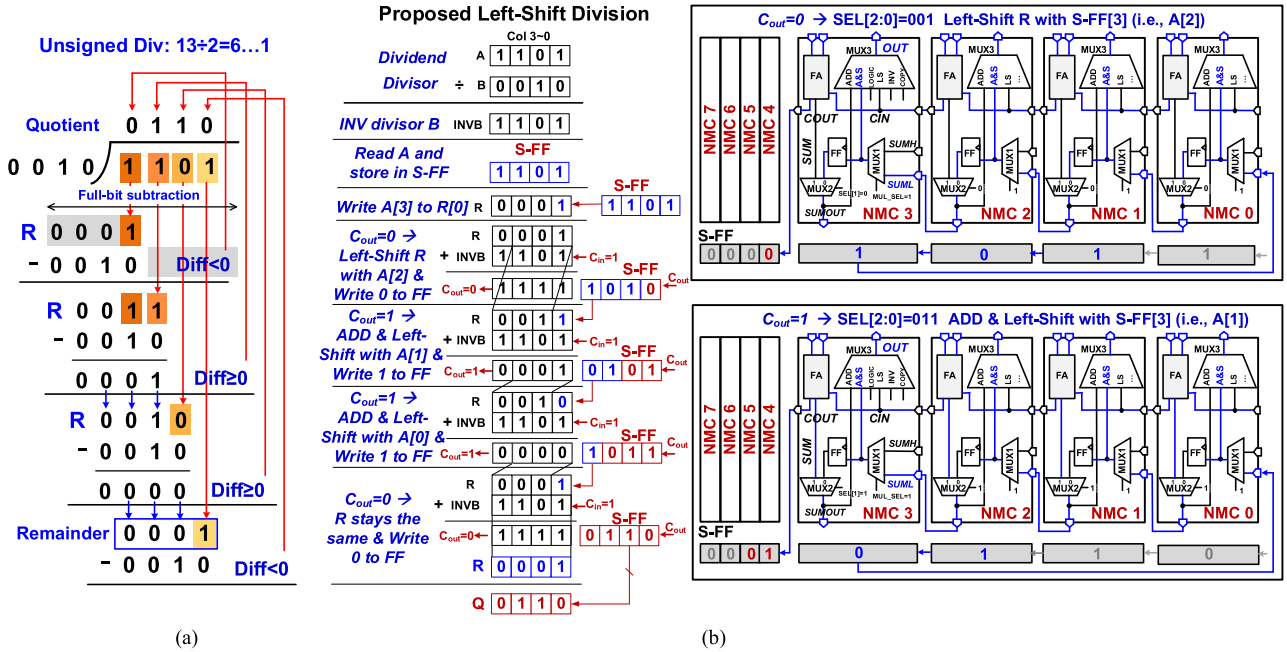


Fig. 14. (a) Conventional unsigned division [32]. (b) Proposed unsigned division flow for supporting DBP data format.

in Fig. 13(b). The computation begins with two initialization cycles to configure S-FF and invert operand A. The subsequent three cycles perform multiplication between signed and unsigned number through iterative right-shift and write-back operations. The sixth cycle completes the signed multiplication by selecting the inverted A to represent the product of A and B [3] and combining it with the accumulated partial sum through ADD and right-shift operation. The half high and low bits of product are therefore, respectively, stored in the NMCs of columns 4–7 and the S-FFs of columns 0–3. Finally, the results are written into the CIM arrays using the diagonal-block storage format.

D. Left-Shift Division Operation

The proposed DBP-CIM supports unsigned division in a left-shift-based manner to reduce the hardware consumption in previous bit-parallel division arithmetic, as shown in Fig. 14. Division algorithm determines each quotient bit by evaluating the subtraction result between the partial remainder and divisor in every cycle, as illustrated in Fig. 14(a). Previous bit-parallel division implementation [32] employs asymmetric padding and shift operations between the dividend and divisor to conduct full-bit-width subtraction for quotient generation, requiring additional NMCs. In contrast, the proposed left-shift division algorithm leverages the frozen low-bit property during decision subtraction, thereby eliminating the need for redundant NMCs with reduced hardware requirement.

The detailed computing flow of left-shift-based division is shown in Algorithm 3. Initially, the result rows of reminder $R[n-1{:}0]$ and quotient $Q[n-1{:}0]$ are set as 0. The operands, dividend $A[n-1{:}0]$ and divisor $B[n-1{:}0]$, are stored in columns 0–$(n-1)$. The S-FF is first initialized by loading $A[n-1{:}0]$. The divisor $B[n-1{:}0]$ is inverted and written into a temporary row as $\mathrm{INVB}[n-1{:}0]$ for preparing the following subtraction-based quotient generation. To start the computing, S-FF$[n-1]$ needs to update the $R[0]$ in the beginning. For the following subsequent computing cycles, the remainder R conducts subtraction with B by reading INVB and R simultaneously for ADD with carry input set as "1." The sum

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:42 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3 Left-Shift-Based Unsigned Division

Input: A[n-1:0], B[n-1:0] // Dividend A and Divisor B
Output: Q[n-1:0], R[n-1:0] //Quotient Q and Remainder R

```
0: Initialization: Store A in S-FFs; Initialize Q and R to 0
1: INVB = INV(B)//Invert divisor B and store INVB in temporary row
2: R = {(n-1)'b0, A[n-1]} //Write A[n-1] to R[0]
3: for (i =0 to n-1) do
4:    {COUT, SUM[n-1:0]}=ADD(INVB,R,1'b1)// Subtraction R-B
5:    S-FF = {S-FF[n-2:0], COUT} // Store Q and A in S-FF
6:    if i == n-1 then //Last computational cycle
7:       if COUT == 1'b0 then //R-B is negative
8:          R=R  //R stays the same
9:       else //R-B is positive
10:         R= SUM[n-1:0] //Write the SUM to R
11:   else
12:      if COUT == 1'b0 then //R-B is negative. S-FF[n-1] = A[n-2-i]
13:         R={R[n-2:0], S-FF[n-1]}//Left shift R with A[n-2-i]
14:      else //R-B is positive
15:         R={SUM[n-2:0], S-FF[n-1]} //Add & left shift with A[n-2-i]
16:   end if
17: end for
18: Return Q, R
```

OUT of subtraction is then left-shifted with S-FF to update reminder R or not, according to the state of carry OUT of subtraction, which stands for the quotient bit in the current cycle. If carry OUT is "0," R–B is negative. The sum OUT of current subtraction is not valid. The remainder of previous cycle is left-shifted with S-FF[$n-1$], ready for subtraction in the next cycle. If carry OUT is "1," R–B is determined as positive. The subtraction result is left-shifted with S-FF[$n-1$] as the least significant bit to update the remainder. Note that, S-FF is consistently left-shifted in both cases, with the least bit S-FF[0] updated by the carry OUT from adjacent division computing column and the highest bit S-FF[$n-1$] popped out to the remainder. For the last computing cycle, if carry OUT of subtraction is "0," the reminder stays the same; otherwise, it will be updated by the sum OUT in current cycle without left-shift operation. Finally, the n-bit remainder and quotient are, respectively, generated in columns 0–($n-1$) and columns n–(2*n−1). The results of remainder and quotient are then written back into CIM array in a diagonal-block format.

The specific example of 4-bit unsigned division between dividend A "1101" and divisor B "0010" is shown in Fig. 14(b). In the initialization stage, A[3:0] is stored into S-FF. The remainder is initialized with R[3:1] being 0 and least significant bit being S-FF [3]. The next three computing cycles repetitively conduct the subtraction between remainder R and divisor B with left-shift operations. The S-FF is left-shifted with carry OUT and pops out the highest significant bit to the remainder in cycles. The subtraction carry OUT is used to determine whether the subtraction result is valid. If the carry OUT is "0," the subtraction in current cycle is considered as invalid. The subtraction result in last cycle that stored in SUMIN_FF is selected and left-shifted with least significant bit being replaced by S-FF [3]. If the carry OUT is "1," the subtraction result of current cycle is accepted and left-shifted with S-FF [3] to update R[3:0]. In the fourth computing cycle, the subtraction without shift operation is conducted to generate the final remainder and quotient. The final results then are generated across eight columns with the remainder stayed in columns 0–3 and quotient shifted into S-FFs of columns 4–7. The reminder and quotient are written back into CIM array in a diagonal manner.

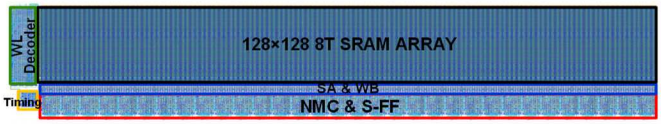


Fig. 15. Layout of the proposed DBP-CIM macro designed in 28-nm CMOS.

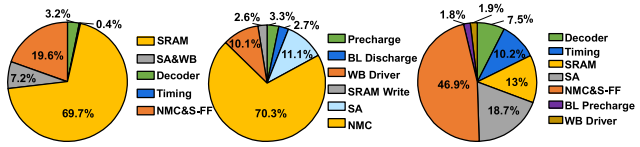


Fig. 16. (a) Macro area breakdown. (b) Cycle latency breakdown. (c) Macro power breakdown. The supply voltage is 0.9 V.

IV. Results and Analysis

This section presents a comprehensive evaluation of the proposed DBP-CIM architecture, assessing its area efficiency, cycle latency, energy consumption, and throughput. The evaluation employs various general-purpose workloads to analyze the performance of the proposed design compared to previous bit-parallel CIM methods. The experimental setup and methodology are introduced in Section IV-A. Section IV-B evaluates the performance of the proposed DBP-CIM macro. The evaluation results for different general-purpose application workloads of the proposed DBP-CIM are presented in Section IV-C.

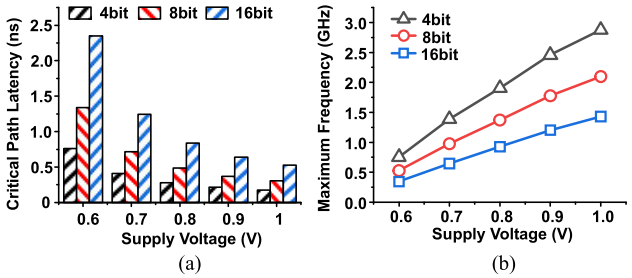
A. Evaluation Setup

To evaluate the performance of the proposed bit-parallel SRAM-CIM, a 16-kb (128 × 128) DBP-CIM macro has been simulated with 28-nm CMOS technology. The analog components, including SRAM cells, SA, and write circuits are designed manually using Cadence Virtuoso. The digital circuit components in DBP-CIM consist of the near-memory computing unit, S-FF, and read/write wordline decoder are implemented in Verilog and synthesis in Synopsys Design Compiler using 28-nm CMOS libraries. IC Compiler tools are also used for the macrolevel physical design. The post-layout parasitic exaction has been carried out for accurate SPICE simulations at 25 °C in TT corner.

B. Evaluation Results of the Proposed DBP-CIM Macro

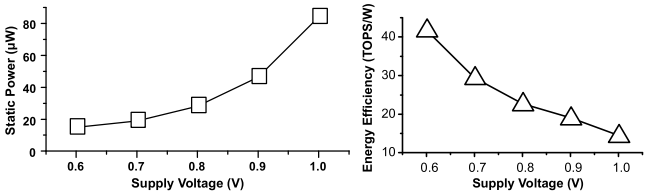
1) Area: The layout of the proposed DBP-CIM macro is shown in Fig. 15. The total macro area, including peripheries, is evaluated as 14005.5 μm^2. The detailed area breakdown of the proposed DBP-CIM macro is shown in Fig. 16(a). The SRAM-CIM cells dominate the area, where the CIM cells in the same row share the RWL and WWL metal layers, while cells in the same column share the source/drain connections to power, ground, read BLs, and write BLs, resulting in an average area of 0.559 μm^2 per 8T SRAM-CIM cell. The precharger is essentially a pair of p-type transistors attached to the BLs and already included in the SRAM array area. The NMC units with extra S-FF occupy 19.6% of total area with the extra S-FF accounting for 1.7% of the total

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:42 UTC from IEEE Xplore. Restrictions apply.



(a) (b)

Fig. 17. (a) Critical path latency of near-memory computing unit under 4-bit, 8-bit, and 16-bit block configurations. (b) Maximum frequency with varied operating voltages and bit width.



(a) (b)

Fig. 18. (a) Static power of the proposed DBP-CIM array and (b) energy efficiency of logic operations across different supplies.

array area. The area overhead of NMC units can be amortized by vertically scaling-up the array size (with increased number of rows) and sharing NMC units across multiple columns through column multiplexing, while the throughput is not influenced. Alternatively, the horizontal scaling-up array size (with increased number of columns) does not reduce the area percentage of NMCs while the throughput is improved.

2) Latency: The cycle latency breakdown under normal supply is shown in Fig. 16(b). The single-cycle delay includes RBL/RBLB charging time, RBL/RBLB discharge delay, SA sensing delay, NMC computation delay, write-back driving delay, and SRAM data flip latency. The near-memory computing unit dominates the latency of single cycle due to the long ripple paths and thus being the critical path of the proposed DBP-CIM. The critical path latency of the proposed DBP-CIM is further evaluated with different operating bit width, as shown in Fig. 17(a). Previous bit-parallel CIM architecture could suffer from frequency degradation as the operating bit width varies throughout a sequential of computing. The proposed DBP-CIM solves this issue by employ a block-based arithmetic with constant operating bit width in a single cycle without affecting the operating frequency. The maximum frequency of different operating bit width is shown in Fig. 17(b). The proposed DBP-CIM achieves 2.097 GHz at supply of 1.0 V for 8-bit block computing. With 4-bit width, the proposed DBP-CIM can achieve up to a frequency of 2.88 GHz.

3) Energy Efficiency and Throughput: The power breakdown of the proposed DBP-CIM is shown in Fig. 16(c). The near-memory computing units and memory I/O modules occupy the major portion of the total power. The static power of the proposed DBP-CIM is shown in Fig. 18(a). The static power is evaluated as 83.96 μW at 1.0 V by disabling all the functional signals, while it achieves a minimal static power of 14.79μ W at 0.6 V. For energy efficiency evaluation, simulations were conducted with the randomly generated operands stored in SRAM. The average energy consumption of various data patterns is taken as the final energy consumption for certain arithmetic operation to evaluate the energy efficiency. The energy efficiency of logic operation is shown in Fig. 18(b). The logic energy efficiency of the proposed DBP-CIM is 14.5 TOPS/W at 1.0 V and the best energy efficiency is achieved as 41.8 TOPS/W at 0.6 V. The evaluation results of 4-bit, 8-bit, and 16-bit energy efficiency and throughput of the proposed DBP-CIM for different arithmetic operations are shown in Figs. 19–21, respectively. The proposed DBP-CIM can support different operating bit width by configuring the connection of NMCs in a minimal granularity of 4. The energy efficiency is increased with decline of supplies, while the throughput is decreased with the reduction of supplies. For 8-bit ADD, the proposed DBP-CIM achieves up to 4.65-TOPS/W energy efficiency at 0.6 V and 31.5 GOPS throughput at 1.0 V. For 8-bit multiplication, the proposed DBP-CIM achieves up to 0.496-TOPS/W energy efficiency at 0.6 V and 3.15 GOPS throughput at 1.0 V.

4) Comparisons With the State-of-the-Art: The detailed comparisons of the proposed DBP-CIM and previous general-purpose SRAM-CIM works are listed in Table III. The proposed DBP-CIM supports general-purpose arithmetic, such as ADD, subtraction, multiplication, and division. Note that, the proposed DBP-CIM architecture can also support signed multiplication by using an XOR gate in each NMC unit for sign extension. The support for signed arithmetic significantly broadens the applicability of the proposed DBP-CIM across diverse applications such as AI inference and digital signal processing, where the operands are typically signed. Besides, compared to previous bit-serial CIM, the proposed DBP-CIM maintains the advantages of low computing complexity, as listed in Table IV.

The proposed DBP-CIM further reduces the usage of both NMC and SRAM-CIM cells by nearly half through diagonal-block-based computing flows. This enables more parallel operations under the same array footprint, resulting in higher throughput for multicycle operations than prior bit-serial and bit-parallel CIM designs. Furthermore, the proposed DBP-CIM shows advantages in the energy efficiency of multicycle operations, which are evident in the energy scaling between the multicycle operations (multiplication and division) and the basic single-cycle operation (ADD). For example, the previous bit-parallel designs [31], [32] require 12× and 18× the energy of an 8-bit ADD to perform 8-bit multiplication. In contrast, the proposed DBP-CIM accomplishes the same operation using less than 10× the energy. This enhancement is primarily attributed to the proposed right-shift-based multiplication, which processes 8-bit instead of 16-bit ADD per cycle, thereby reducing computing energy without compromising throughput. Similarly, the proposed left-shift-based division can also substantially improve the energy efficiency compared to [32], as discussed in Section III-D. Therefore, these advancements collectively establish DBP-CIM as a superior solution for general-purpose CIM applications, offering both performance and efficiency gains over previous bit-serial and bit-parallel CIM architectures.

C. Benchmark on General Tasks

To demonstrate the generality and efficiency of the proposed DBP-CIM, the general-purpose benchmarks across various

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:42 UTC from IEEE Xplore. Restrictions apply.

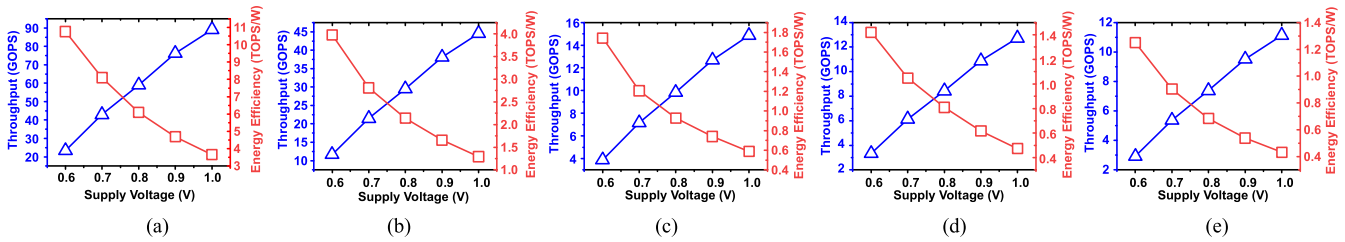


Fig. 19. 4-bit energy efficiency and throughput of the proposed DBP-CIM for (a) ADD, (b) subtraction, (c) unsigned multiplication, (d) signed multiplication, and (e) division with varied operating voltages.

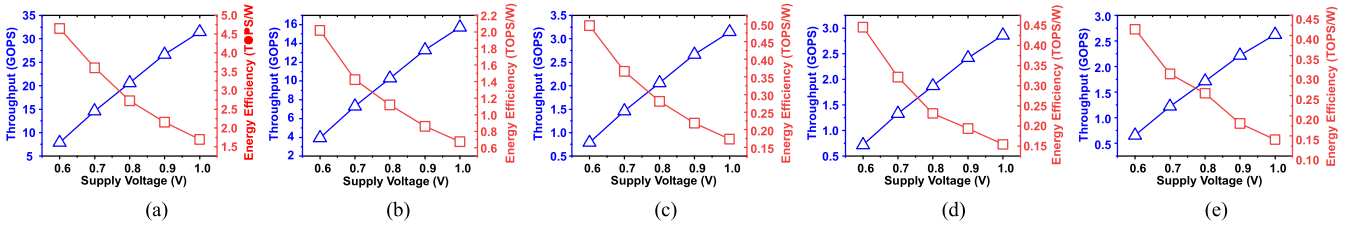


Fig. 20. 8-bit energy efficiency and throughput of the proposed DBP-CIM for (a) ADD, (b) subtraction, (c) unsigned multiplication, (d) signed multiplication, and (e) division with varied operating voltages.

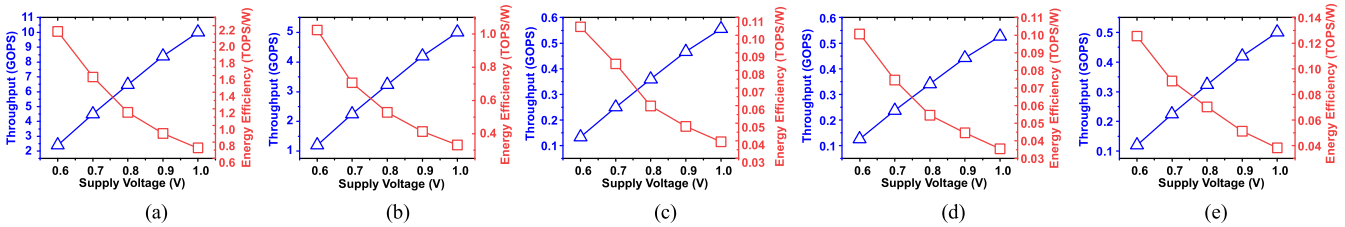


Fig. 21. 16-bit energy efficiency and throughput of the proposed DBP-CIM for (a) ADD, (b) subtraction, (c) unsigned multiplication, (d) signed multiplication, and (e) division with varied operating voltages.

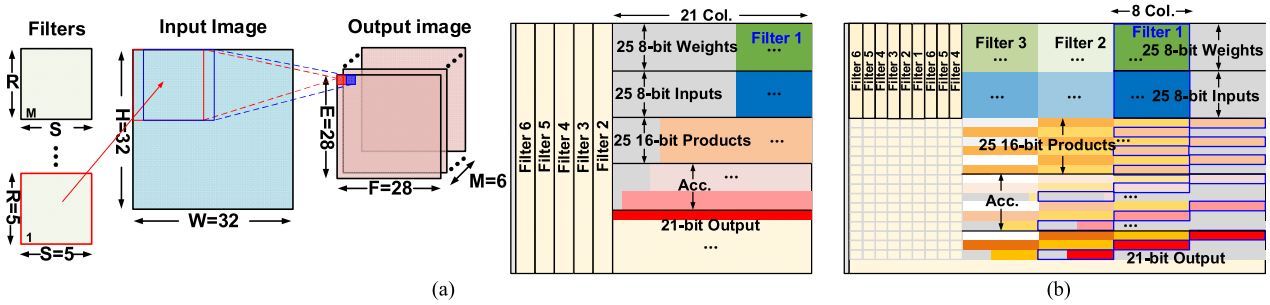


Fig. 22. Examples of mapping the MAC-based convolution layer into (a) conventional bit-parallel CIM array and (b) proposed DBP-CIM array.

applications are selected for evaluations, such as AI inference, image processing, signal processing, and image encryption. The diverse general-purpose workloads can be efficiently supported by decomposing them into groups of computing tasks, each consisting solely of basic arithmetic or logic operations supported by the proposed DBP-CIM. The specific mapping examples of MAC-based application for AI inference (convolution) are selected to demonstrate the task mapping of the proposed DBP-CIM, as illustrated in Fig. 22. Both inputs and weights are needed to map into the CIM array, where the input is a 32 $\times$ 32 single-channel (C = 1) image and the weights consists of six convolutional kernels (M = 6), each with a 5 $\times$ 5 (R$\times$S) receptive field and a stride of 1, producing six OUT feature maps of size 28 $\times$ 28 ($E \times F$). Thus, this layer requires (28 $\times$ 28 $\times$ 6) groups of MAC operations, where each MAC involves 5 $\times$ 5 multiplications between inputs and weights and their subsequent accumulation. With 8-bit quantized inputs and weights, each multiplication yields a 16-bit product, and the accumulation of 25 products generates in a 21-bit MAC result. In the baseline design, using conventional bit-parallel data mapping, a 128 $\times$ 128 SRAM-CIM array can only compute six MAC groups in parallel, with each OUT channel assigned to 21 columns, as shown in Fig. 22(a). In contrast, the proposed DBP-CIM, leveraging the diagonal-data mapping, supports simultaneous processing of up to 14 MAC groups within a single array. In the evaluated CONV application, by duplicating the weights of all six OUT channels, the proposed DBP-CIM can process 12 MAC groups

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:42 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPARISON WITH PRIOR GENERAL-PURPOSE SRAM-CIM WORKS

		JSSC'20 [29]	ESSCRIC'22 [30]	DAC'20 [31]	TVLSI'25 [22]	TCAS-I'23 [32]	**This Work**
Technology Node (nm)		28	65	28	40	65	**28**
Implementations		Silicon Test	Silicon Test	Post-layout Simulation	Post-layout Simulation	Silicon Test	**Post-layout Simulation**
SRAM-CIM Cell Structure		8T	8T	6T	10T	8T	**8T**
Supply (V)		0.6-1.1	0.8-1.2	0.6-1.1	1.1	0.7-1.2	**0.6-1.0**
Max Frequency (MHz)		475 (1.1V)	1230 (1.2V)	372 (0.6V) 2250 (1.1V)	752 (1.1V)	160 (1.0V)	**2097 (1.0V)** **525 (0.6V)**
Array Size		128×256	128×128	128×128	64×64	256×64	**128×128**
Bit-Cell Area (μm^2)		0.78	4.56	NR	2.59	1.57	**0.559**
Macro Area (mm^2)		0.038	0.082	NR	0.02	0.063	**0.014**
Computing Scheme		Bit-serial	Bit-serial	Bit-parallel	Bit-wise	Bit-parallel	**Bit-parallel/ Diagonal-block parallel**
Supported Function[++]		Add/Sub/ UMult/UDiv/Logic	Add/Sub/ UMult/Logic	Add/Sub/ UMult/Logic	Logic/CAM	Add/Sub/UMult/ UDiv/Logic/CAM	**Add/Sub/UMult/ SMult/UDiv/Logic**
Logic Energy Efficiency (TOPS/W)		NR	34.98	NR	34.2	NR	**41.8[*] (0.6V)** **14.5[*] (1.0V)**
Add (8-bit)	Throughput (GOPS)	3.8 (1.1V)	17.6[+] (1.2V)	5.95 (0.6V) 26.53 (0.9V)	NS	2.24 (1.2V)	**7.88 (0.6V)** **31.5 (1.0V)**
	Energy Efficiency (TOPS/W)	5.27 (0.6V)	NR	8.09 (0.6V) 3.64 (0.9V)		3.11 (0.7V)	**4.65 (0.6V)** **1.69 (1.0V)**
Sub (8-bit)	Throughput (GOPS)	1.9 (1.1V)	9.3[+] (1.2V)	NR	NS	NR	**3.94 (0.6V)** **15.7 (1.0V)**
	Energy Efficiency (TOPS/W)	NR	NR				**2.02 (0.6V)** **0.673 (1.0V)**
UMult (8-bit)	Throughput (GOPS)	0.29 (1.1V)	1.5 (1.2V)	0.33 (0.6V) 1.47 (0.9V)	NS	0.13 (1.2V)	**0.79 (0.6V)** **3.15 (1.0V)**
	Energy Efficiency (TOPS/W)	0.55 (0.6V)	NR	0.68 (0.6V) 0.29 (0.9V)		0.17 (0.7V)	**0.496 (0.6V)** **0.174 (1.0V)**
SMult (8-bit)	Throughput (GOPS)	NS	NS	NS	NS	NS	**0.72 (0.6V)** **2.86 (1.0V)**
	Energy Efficiency (TOPS/W)						**0.445 (0.6V)** **0.154 (1.0V)**
UDiv (8-bit)	Throughput (GOPS)	0.22 (1.1V)	NS	NS	NS	0.09 (1.2V)	**0.66 (0.6V)** **2.62 (1.0V)**
	Energy Efficiency (TOPS/W)	NR				0.127 (0.7V)	**0.415 (0.6V)** **0.141 (1.0V)**

NR: Not Reported. NS: Not Supported. *: Include write back energy. [+]: Scaled to 8-bit operation. [++]: Signed multiplication is supported in this work, but not supported in previous work.
Add: Addition. Sub: Subtraction. UMult: Unsigned multiplication. SMult: Signed multiplication. UDiv: Unsigned division.

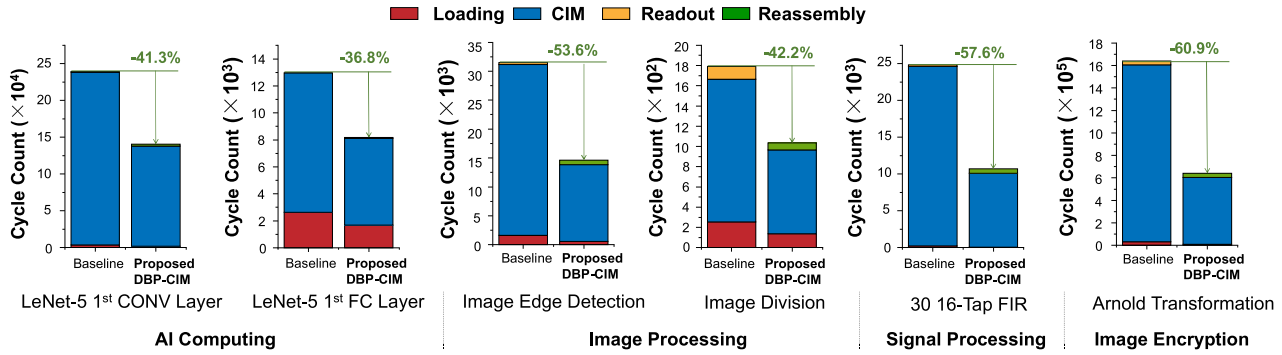


Fig. 23. Cycle count evaluations of previous bit-parallel CIM baseline and the proposed DBP-CIM (considering the reassembly unit) for various workloads.

TABLE IV
OPERATING COMPLEXITY FOR N-BIT ARITHMETIC OPERATIONS

	Add	Sub	UMult	SMult	UDiv
Bit-Serial [29]	n+1	2n+1	n^2+5n-2	-	1.5n^2+5.5n
Bit-Parallel [32]	1	2	n+2	-	n+5
Proposed DBP	**1**	**2**	***n*+2**	***n*+3**	***n*+4**

in parallel, significantly accelerating the convolution for each OUT feature map as shown in Fig. 22(b). Note after each MAC computation, the input data for the next sliding window must be updated. This process repeats until all OUT feature maps are computed.

For a fair comparison, the CIM baseline follows the same design rules and shares the same SRAM-CIM cell and peripheral circuits as the proposed DBP-CIM, while the near-memory computing unit is modified for conventional bit-parallel arithmetic operations [31], [32]. The evaluation results of cycle count and energy consumption of previous bit-parallel CIM baseline and the proposed DBP-CIM are shown in Figs. 23 and 24, respectively. For the cycle count analysis,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:42 UTC from IEEE Xplore. Restrictions apply.

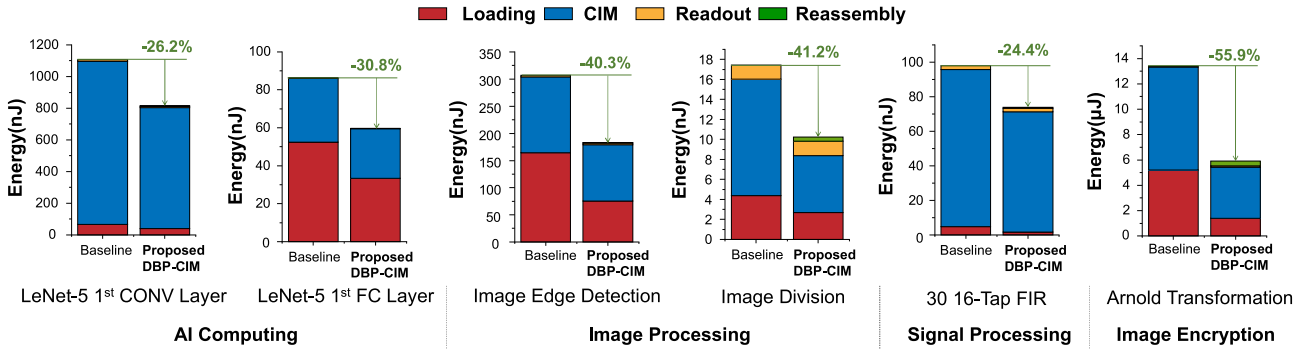


Fig. 24. Energy consumption evaluations of previous bit-parallel CIM baseline and the proposed DBP-CIM (considering the reassembly unit) for various workloads.

the proposed DBP-CIM reduces the loading and computing cycles of previous bit-parallel CIM baseline across all the evaluated workloads, achieving up to 60.9% cycle reduction. This is attributed to the use of the proposed diagonal-block mapping scheme, which increases the computing parallelism. Besides, the operands do not need to be loaded into CIM array with padded dummy zeros, which significantly reduces the loading cycles.It should be noted that the reassembly operation of the proposed DBP-CIM has little impact on overall system performance, as it only constitutes a small portion of the total cycle count. Furthermore, the proposed DBP-CIM lowers the computing and data loading energy across different workloads, achieving up to 55.9% energy savings over previous bit-parallel CIM baseline, thanks to the proposed diagonal-block mapping and the associated arithmetic flows.

V. Conclusion

This article presents an energy-efficient bit-parallel SRAM-CIM with the compact DBP data mapping method and optimized arithmetic flows, which improves the hardware utilization and enhances the computing parallelism of previous bit-parallel CIM architecture. The post-layout simulations on 28-nm CMOS technology show that the proposed DBP mapping-based SRAM-CIM achieves higher energy efficiency and throughput for general-purpose arithmetic operations, compared with state-of-the-art works. Furthermore, evaluations on the diverse general-purpose benchmarks demonstrate that the proposed method reduces energy consumption and computing cycles by up to 55.9% and 60.9%, respectively, compared to the conventional bit-parallel CIM method.

References

[1] A. Sebastian, M. Le Gallo, R. Khaddam-Aljameh, and E. Eleftheriou, "Memory devices and applications for in-memory computing," *Nature Nanotechnol.*, vol. 15, no. 7, pp. 529–544, Jul. 2020.

[2] D.-Q. You et al., "A 22 nm 104.5TOPS/W μ-NMC-Δ-IMC heterogeneous STT-MRAM CIM macro for noise-tolerant Bayesian neural networks," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2025, pp. 1–3.

[3] G. Yin et al., "Cramming more weight data onto compute-in-memory macros for high task-level energy efficiency using custom ROM with 3984-kb/mm^2 density in 65-nm CMOS," *IEEE J. Solid-State Circuits*, vol. 59, no. 6, pp. 1912–1925, Jun. 2024.

[4] Z. Zhou et al., "ArPCIM: An arbitrary-precision analog computing-in-memory accelerator with unified INT/FP arithmetic," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 5, pp. 2229–2242, May 2025.

[5] Y. Sun, Z. Li, W. Liu, W. He, Q. Wang, and Z. Mao, "BC-MVLiM: A binary-compatible multi-valued logic-in-memory based on memristive crossbars," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 5, pp. 2048–2061, May 2023.

[6] A. Fathollahi, A. Amirany, and M. H. Moaiyeri, "Ternary computing using a novel spintronic multi-operator logic-in-memory architecture," *Results Eng.*, vol. 25, pp. 1–17, Mar. 2025.

[7] Z. Chen et al., "PICO-RAM: A PVT-insensitive analog compute-in-memory SRAM macro with in situ multi-bit charge computing and 6T thin-cell-compatible layout," *IEEE J. Solid-State Circuits*, vol. 60, no. 1, pp. 308–320, Jan. 2025.

[8] P. Chen et al., "A 22-nm delta-sigma computing-in-memory SRAM macro with near-zero-mean outputs and LSB-first ADCs for edge AI processing," *IEEE J. Solid-State Circuits*, vol. 60, no. 8, pp. 3020–3032, Aug. 2025.

[9] Y. Sun et al., "CREAM: Computing in ReRAM-assisted energy- and area-efficient SRAM for reliable neural network acceleration," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 8, pp. 3198–3211, Aug. 2023.

[10] W. Lu et al., "High-reliability and high-throughput CIM 10T-SRAM for multiplication and accumulation operations with 274.3 GOPS and 200–237.5 TOPS/W," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 33, no. 4, pp. 1073–1081, Apr. 2025.

[11] Y. Wang et al., "PIPECIM: Energy-efficient pipelined computing-in-memory computation engine with sparsity-aware technique," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 33, no. 2, pp. 525–536, Feb. 2025.

[12] X. Xiang, Z. Yue, X. Zhang, S. Wei, Y. Hu, and S. Yin, "Dyn-bitpool: A 28 nm 27 TOPS/W two-sided sparse CIM accelerator featuring a balanced workload scheme and high CIM macro utilization," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 5, pp. 2216–2228, May 2025.

[13] H. You, W. Li, D. Shang, Y. Zhou, and S. Qiao, "A 1–8b reconfigurable digital SRAM compute-in-memory macro for processing neural networks," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 4, pp. 1602–1614, Apr. 2024.

[14] H. Kim, J. Mu, C. Yu, T. T.-H. Kim, and B. Kim, "A 1–16b reconfigurable 80Kb 7T SRAM-based digital near-memory computing macro for processing neural networks," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 4, pp. 1580–1590, Apr. 2023.

[15] H. Kim, T. Yoo, T. T.-H. Kim, and B. Kim, "Colonnade: A reconfigurable SRAM-based digital bit-serial compute-in-memory macro for processing neural networks," *IEEE J. Solid-State Circuits*, vol. 56, no. 7, pp. 2221–2233, Jul. 2021.

[16] C. Eckert et al., "Neural cache: Bit-serial in-cache acceleration of deep neural networks," in *Proc. ACM/IEEE 45th Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2018, pp. 383–396.

[17] X. Wang et al., "A 28 nm 17.83-to-62.84TFLOPS/W broadcast-alignment floating-point CIM macro with non-two's-complement MAC for CNNs and transformers," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2025, pp. 254–256.

[18] P.-C. Wu et al., "A floating-point 6T SRAM in-memory-compute macro using hybrid-domain structure for advanced AI edge chips," *IEEE J. Solid-State Circuits*, vol. 59, no. 1, pp. 196–207, Jan. 2024.

[19] J. Yue et al., "A 28 nm 16.9–300TOPS/W computing-in-memory processor supporting floating-point NN inference/training with intensive-CIM sparse-digital architecture," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 1–3.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:42 UTC from IEEE Xplore. Restrictions apply.

[20] S. Jeong, J. Park, and D. Jeon, "A 28 nm 1.644TFLOPS/W floating-point computation SRAM macro with variable precision for deep neural network inference and training," in *Proc. IEEE 48th Eur. Solid State Circuits Conf. (ESSCIRC)*, Sep. 2022, pp. 145–148.
[21] Y. Wang et al., "A 28 nm 83.23TFLOPS/W POSIT-based compute-in-memory macro for high-accuracy AI applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 566–568.
[22] Z. Zhang, Z. Chen, J. Wang, G. Xie, and G. Liu, "Reconfigurable 10T SRAM for energy-efficient CAM operation and in-memory computing," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 33, no. 4, pp. 1065–1072, Apr. 2025.
[23] Y. Zhang, L. Xu, Q. Dong, J. Wang, D. Blaauw, and D. Sylvester, "Recryptor: A reconfigurable cryptographic cortex-M0 processor with in-memory and near-memory computing for IoT security," *IEEE J. Solid-State Circuits*, vol. 53, no. 4, pp. 995–1005, Apr. 2018.
[24] D. Reis, H. Geng, M. Niemier, and X. S. Hu, "IMCRYPTO: An in-memory computing fabric for AES encryption and decryption," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 30, no. 5, pp. 553–565, May 2022.
[25] Z. Lin et al., "In-memory computing with double word lines and three read ports for four operands," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 5, pp. 1316–1320, May 2020.
[26] S. Aga et al., "Compute caches," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, Feb. 2017, pp. 481–492.
[27] Y. Ju, Y. Wei, and J. Gu, "A 65 nm general-purpose compute-in-memory processor supporting both general programming and deep learning tasks," *IEEE J. Solid-State Circuits*, vol. 60, no. 4, pp. 1500–1511, Apr. 2025.
[28] A. K. Ramanathan et al., "CiM3D: Comparator-in-memory designs using monolithic 3-D technology for accelerating data-intensive applications," *IEEE J. Explor. Solid-State Comput. Devices Circuits*, vol. 7, pp. 79–87, 2021.
[29] J. Wang et al., "A 28-nm compute SRAM with bit-serial logic/arithmetic operations for programmable in-memory vector computing," *IEEE J. Solid-State Circuits*, vol. 55, no. 1, pp. 76–86, Jan. 2020.
[30] A. Sridharan, S. Angizi, S. K. Cherupally, F. Zhang, J.-S. Seo, and D. Fan, "A 1.23-GHz 16-kb programmable and generic processing-in-SRAM accelerator in 65 nm," in *Proc. IEEE 48th Eur. Solid State Circuits Conf. (ESSCIRC)*, Sep. 2022, pp. 153–156.
[31] K. Lee, J. Jeong, S. Cheon, W. Choi, and J. Park, "Bit parallel 6T SRAM in-memory computing with reconfigurable bit-precision," in *Proc. 57th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2020, pp. 1–6.
[32] Y. Chen, J. Mu, H. Kim, L. Lu, and T. T. Kim, "BP-SCIM: A reconfigurable 8T SRAM macro for bit-parallel searching and computing in-memory," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 5, pp. 2016–2027, May 2023.
[33] W. A. Simon, Y. M. Qureshi, M. Rios, A. Levisse, M. Zapater, and D. Atienza, "BLADE: An in-cache computing architecture for edge devices," *IEEE Trans. Comput.*, vol. 69, no. 9, pp. 1349–1363, Sep. 2020.
[34] A. Agrawal, M. Ali, M. Koo, N. Rathi, A. Jaiswal, and K. Roy, "IMPULSE: A 65-nm digital compute-in-memory macro with fused weights and membrane potential for spike-based sequential learning tasks," *IEEE Solid-State Circuits Lett.*, vol. 4, pp. 137–140, 2021.
[35] A. K. Rajput and M. Pattanaik, "Energy efficient 9T SRAM with R/W margin enhanced for beyond von-neumann computation," in *Proc. 24th Int. Symp. VLSI Design Test (VDAT)*, Jul. 2020, pp. 1–4.
[36] N. S. Kim, D. Chen, J. Xiong, and W.-M.-W. Hwu, "Heterogeneous computing meets near-memory acceleration and high-level synthesis in the post-Moore era," *IEEE Micro*, vol. 37, no. 4, pp. 10–18, Aug. 2017.
[37] A. Amirany and R. Rajaei, "Low power, and highly reliable single event upset immune latch for nanoscale CMOS technologies," in *Proc. Electr. Eng. (ICEE)*, Jul. 2018, pp. 103–107.



Dengfeng Wang (Student Member, IEEE) received the B.E. degree from the Department of Physics and Astronomy, Shanghai Jiao Tong University, Shanghai, China, in 2020, where he is currently working toward the Ph.D. degree at the Department of Micro-Nano Electronics.

His research mainly focuses on computing-in-memory and monolithic 3-D circuit design.



Chengjun Chang (Student Member, IEEE) received the B.S. degree in microelectronics science and engineering and the M.S. degree in electronic science and technology from Shanghai Jiao Tong University, Shanghai, China, in 2022 and 2025, respectively.

Her research interests include in-memory computing and digital circuit design.



Weifeng He (Senior Member, IEEE) received the B.S., M.S., and Ph.D. degrees in microelectronics and solid-state electronics from Harbin Institute of Technology, Harbin, China, in 1999, 2001, and 2005, respectively.

He is currently a Full Professor with the Department of Micro-Nano Electronics, Shanghai Jiao Tong University, Shanghai, China. His research interests include low-power circuits design, VLSI architecture for video signal processing, and computer system architecture.



Guanghui He (Member, IEEE) received the B.S. degree in electronic engineering from the University of Electronic Science and Technology of China, Chengdu, China, in 2002, and the Ph.D. degree in electronic engineering from Tsinghua University, Beijing, China, in 2007.

Currently, he is a Professor with the Department of Micro-Nano Electronics and the MoE Key Laboratory of Artificial Intelligence, Shanghai Jiao Tong University, Shanghai, China. His research interests include energy-efficient algorithms and circuit design for wireless communication, image processing, emerging memory devices, and artificial intelligent systems.



Yanan Sun (Senior Member, IEEE) received the B.E. degree in microelectronics from Shanghai Jiao Tong University, Shanghai, China, in 2009, and the Ph.D. degree from The Hong Kong University of Science and Technology, Hong Kong, in 2015.

She is currently an Associate Professor with the Department of Micro-Nano Electronics, Shanghai Jiao Tong University. Her research area is energy-efficient VLSI circuit and system design with emerging nanotechnologies.

Dr. Sun received the Best Paper Award Nomination in 2020 IEEE Design, Automation, and Test in Europe Conference (DATE), the Best Paper Award Nomination in 2023 IEEE International Conference on Integrated Circuits, Technologies and Applications (ICTA), and the Best Paper Award (First Place) in 2014 IEEE International Conference on Microelectronics (ICM). She currently serves on the Editorial Board of *Microelectronics Journal, Integrated Circuits and Systems*.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:42 UTC from IEEE Xplore. Restrictions apply.